Automated Warehouse Sorting System - Mastery Roadmap

Overview

This roadmap guides you through the **Automated Warehouse Sorting System** project in a **structured, project-driven** manner. Each phase introduces key PLC concepts as the system progresses towards full automation. An **Alarm Manager** will be developed throughout the project to handle errors and system diagnostics.

🗓️ Project Timeline & Learning Plan (4 Months)

Phase 1: System Architecture & Initial Setup (Weeks 1-2)

🔹 Concepts Covered

✅ Hardware Selection: PLC, Motors, Sensors, Actuators
✅ Conveyor System Design & Basic Control
✅ Timers & Counters for Initial Movement Logic
✅ State Machine Basics (Idle, Running, Stopped)
✅ Initial Alarm Manager Design

🛠️ Tasks & Exercises

- Set up **TwinCAT3 environment** and configure I/O mapping
  - 🔹 **Hint:** Verify hardware connections using test signals before proceeding.
- Implement **basic conveyor control** (Start/Stop using push buttons)
  - 🔹 **Hint:** Use a simple ladder logic rung for initial testing before integrating structured text.
- Configure **Timers & Counters** to track item movement
  - 🔹 **Hint:** Use a TON (On-Delay Timer) for smooth startup of the conveyor.
- Implement **basic state handling** for conveyor movement
  - 🔹 **Hint:** Start with a three-state machine (Idle, Running, Stopped) and expand later.
- Develop **Alarm Manager function block** (logs alarms, resets errors)
  - 🔹 **Hint:** Store alarm logs in an array for future retrieval.

Phase 2: Object-Oriented Programming (OOP) in PLC (Weeks 3-4)

🔹 Concepts Covered

✅ Function Blocks for Modular Design
✅ Encapsulation & Reusability
✅ Advanced State Handling
✅ Error Handling: Sensor Failures, Emergency Stop
✅ Expanding the Alarm Manager

🛠️ Tasks & Exercises

- Create **Function Blocks** for Conveyor, Motor, Sensors
  - 🔹 **Hint:** Each function block should include Start, Stop, Status, and Fault outputs.
- Implement a **state machine** for sorting logic
  - 🔹 **Hint:** Use ENUM types for state representation to enhance readability.
- Add **error handling** for conveyor overload & sensor failures
  - 🔹 **Hint:** Implement a debounce timer for sensor signals to filter noise.
- Write & test **OOP-based motion logic** for conveyor belt
  - 🔹 **Hint:** Use methods inside function blocks to encapsulate repetitive logic.
- Expand **Alarm Manager** to log sensor failures & emergency stops
  - 🔹 **Hint:** Use a FIFO buffer to store recent alarms for quick diagnosis.

Phase 3: Scanning & Identification System (Weeks 5-6)

🔹 Concepts Covered

✅ Integrating Barcode/RFID Scanner with PLC
✅ Data Handling & Storage
✅ Communication Between Scanner & Sorting Logic
✅ HMI Display for Scanned Items
✅ Alarm Manager Updates for Data Errors

🛠️ Tasks & Exercises

- Configure **RFID/Barcode Scanner** with TwinCAT3
  - 🔹 **Hint:** Use a separate task cycle for scanner processing to avoid PLC scan time issues.
- Implement **Data Handling Logic** to store & process package IDs
  - 🔹 **Hint:** Store package IDs in a structured array or a linked list.
- Create an **HMI screen** to display scanned package details
  - 🔹 **Hint:** Use color coding for different package categories for easy identification.
- Develop **decision logic** for sorting based on scanned data
  - 🔹 **Hint:** Implement a lookup table to match scanned IDs to sorting destinations.
- Enhance **Alarm Manager** to detect and report scanner errors
  - 🔹 **Hint:** Generate a warning if the scanner fails multiple consecutive scans.

Phase 4: Sorting & Motion Control (Weeks 7-8)

🔹 Concepts Covered

✅ Servo/Stepper Motor Control
✅ Positioning & Sorting Logic
✅ PID Tuning for Precise Movements
✅ Re-Sorting & Handling Incorrect Packages
✅ Alarm Handling for Motion Control

🛠️ Tasks & Exercises

- Implement **servo control for sorting arms**
  - 🔹 **Hint:** Use a position feedback loop for accuracy.
- Apply **PID tuning** for accurate positioning
  - 🔹 **Hint:** Start with conservative gains and incrementally tune for stability.
- Develop logic to **route packages based on scanned data**
  - 🔹 **Hint:** Implement a state machine to track package movement and sort decisions.
- Implement **misplaced package detection & re-sorting mechanism**
  - 🔹 **Hint:** Use a timeout function to detect unclaimed packages.
- Expand **Alarm Manager** to track motor overload & positioning failures
  - 🔹 **Hint:** Include a warning if position deviation exceeds a set threshold.

Phase 5: Advanced Communication & Remote Monitoring (Weeks 9-10)

🔹 Concepts Covered

✅ OPC UA Communication with External Database
✅ Real-Time Monitoring & Alerts
✅ Remote Manual Override via HMI
✅ Advanced Alarm Reporting & Notifications

🛠️ Tasks & Exercises

- Establish **OPC UA communication** for database integration
  - 🔹 **Hint:** Use separate tasks for real-time and historical data updates.
- Implement **real-time error & system alerts**
  - 🔹 **Hint:** Use event-driven updates instead of polling for better performance.
- Add a **manual override option** on HMI
  - 🔹 **Hint:** Restrict manual controls to maintenance mode only.

- Configure **historical data logging for diagnostics**
  - 🔹 **Hint:** Store logs in a structured CSV format for easy analysis.
- Implement **Alarm Manager notification system via OPC UA**
  - 🔹 **Hint:** Send critical alarms to remote clients via MQTT.

Phase 6: Optimization & Best Practices (Weeks 11-12)

🔹 Concepts Covered

✅ Performance Optimization
✅ Industrial Safety & Redundancy
✅ Advanced Data Logging & Reporting
✅ Final Project Deployment & Documentation
✅ Alarm System Finalization

🛠️ Tasks & Exercises

- Optimize sorting logic for **faster cycle times**
  - 🔹 **Hint:** Reduce scan cycle time by optimizing ladder logic execution order.
- Implement **fail-safe mechanisms** (redundancy, emergency stops)
  - 🔹 **Hint:** Use a watchdog timer to detect unexpected system freezes.
- Finalize **HMI with real-time system overview & reports**
  - 🔹 **Hint:** Implement a user role system for secure access control.
- Document **final project code, structure & troubleshooting guide**
  - 🔹 **Hint:** Include a detailed failure mode analysis in documentation.
- Complete **Alarm Manager with system-wide logging & diagnostics**
  - 🔹 **Hint:** Implement auto-reset logic for non-critical alarms.

📂 Deliverables & Learning Outcomes

✅ **Fully Functional Automated Warehouse Sorting System**
✅ **Optimized & Modular PLC Codebase (Structured Text - ST)**
✅ **HMI with Real-Time Monitoring & Controls**
✅ **Industry-Ready Documentation & Performance Reports**
✅ **Comprehensive Alarm Manager for Fault Detection & Troubleshooting**

🎯 **Ready to build an industry-level sorting system? Let's get started!** 🚀